

Claude Design — what we learned building the Codex deck

Internal share-out from the Codex deck experiment. We used Claude Design for the design system and early prototype, then handed off to Claude Code for the build. Here's the playbook, the gotchas, and the call on when to use which.

May 2026 · Adrianto · Nanovest engineering

TL;DR — the playbook

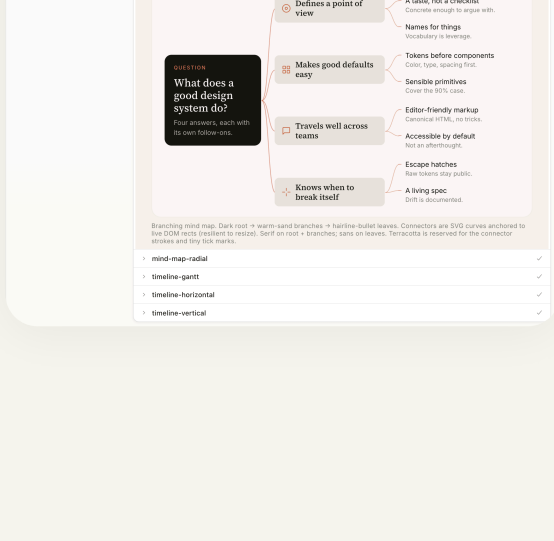
- ▶ **Design system first.** Build it, review it, mark good/rework — then generate anything else.
 - ▶ **One task per session.** The token compounding effect is real; splitting sessions cuts 60–80% of per-message cost.
 - ▶ **Watch the weekly limit.** Claude Design has its own pool, separate from Claude Code / claude.ai chat.
 - ▶ **Hand off early.** Claude Design for prototype-grade work; Claude Code for the actual build.
- Strategic recommendation:** use Claude Design for the design system + a near-final prototype, then move to Claude Code. Don't try to ship from Claude Design alone.

A. What Claude Design is good at

Capabilities we used and would recommend exploring for any team starting out.

Design system intake & visualisation

Claude Design ingests Figma files, code repos, and reference assets to build a design system you can review before any artefact is generated. The review is conversational rather than a formal "approve / reject" UI — but it visualises tokens, components, and typography clearly enough that you can mark good vs. needs-rework as you iterate.

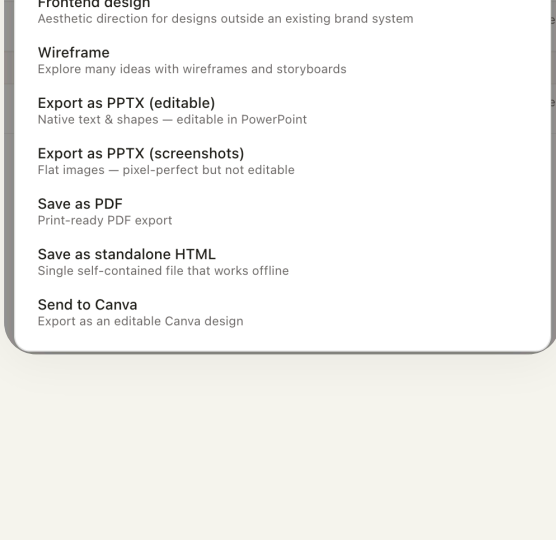
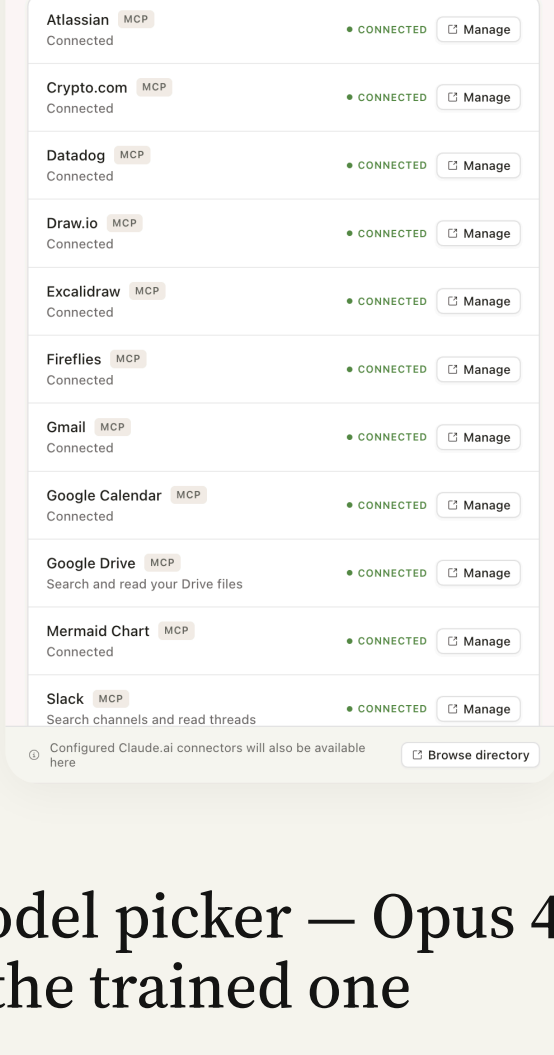


Comment, Edit, Draw, Tweak

Four in-canvas affordances. *Comment* and *Edit* are familiar; *Draw* lets you sketch on the artefact; *Tweak* is the standout — sliders, colour pickers, and parameterised controls let you refine without re-prompting the model. Tweak is where Claude Design feels most like a real design tool.

MCP connectors and predefined skills

Your claude.ai-configured connectors (Slack, Notion, Asana, Microsoft 365, etc.) and predefined skills (e.g. frontend-design) are reachable from inside a Claude Design session. The docs don't enumerate this clearly, but in practice it works — the canvas can call out to Slack to read a thread, or invoke a skill recipe mid-generation.



Model picker — Opus 4.7 is the trained one

Claude Design exposes a model picker (Haiku 4.5, Opus 4.7, Sonnet 4.6, Opus 4.6, Opus 3, Sonnet 4.5). The default — and the one Anthropic actually trained for design+code generation — is **Opus 4.7**. Other models work, but produce noticeably weaker visual output. So the decision is made for you: stay on the default.

B. Best practices we'd repeat

Four habits that compounded into faster, cleaner output.

1. Design system first, generation second

Don't let Claude Design start producing artefacts until the system (tokens, type, components, voice) is reviewed and locked. Skipping this step means every later generation drifts a little, and you end up paying tokens to fix consistency that should have been free.

2. One task per session

The "token compounding effect" is measurable: cumulative session cost can hit 250K+ tokens by the 30th message in a long session. Splitting into per-task sessions (with a short summary carry-over) cuts 60–80% of per-message cost. The cost discipline matters because of point 3.

3. Update the result *after* tweaks, before handoff

In our build, applying tweaks didn't always propagate into the underlying HTML output that handoff would carry forward. Fix: explicitly ask Claude Design to "update the result based on the tweaks I just applied" before moving to handoff. Worth verifying — it may be path-dependent — but the check is cheap and the failure mode (broken WYSIWYG) is expensive.

4. Plan for Claude Code as a fallback

Claude Design and Claude Code have separate weekly limits. If you cap out Design, the build can continue in Code — but Code's limit is *shared* with claude.ai chat, so the fallback only works if you haven't already burned that pool. Build the budget plan up front: how much do you want left in Code when Design is done?

C. Gotchas — what bit us

Each gotcha pairs our direct observation with what external research could (or couldn't) confirm. Where the two diverge, we say so.

Weekly limits hit fast — and they're separate

What we saw: token usage felt high; we hit caps quickly. **What research confirmed:** Claude Design has a separate weekly limit pool from Claude Code / claude.ai chat. The March 2026 2x bonus excluded Design specifically, suggesting it's metered distinctly. Plan accordingly.

PPTX export is lossy (PDF is fine)

What we saw: built-in export to PowerPoint produced output that didn't match the on-screen design. **What research confirmed:** PPTX export is documented as lossy — gradients don't render, fonts shift, image crops differ. PDF export is reliable. If the destination is a slide deck, expect to rebuild in PowerPoint or render via your own pipeline.

The main HTML in handoff is bulky

What we saw: when you hand off to Claude Code, the project includes multiple files (HTML, CSS, assets) — but the main HTML deliverable is a single bulky file rather than a component-decomposed structure. Reading or editing it in Claude Code burns context. **How to think about it:** Claude Design is for prototype-grade work. The bulky-main-HTML is acceptable for prototypes but doesn't scale to a complex app without manual decomposition in Claude Code.

Tweaks can clutter the handoff output

What we saw: tweak metadata (or the styles tweaks generated) persisted into the handoff HTML, where it's painful to clean up. **What research could confirm:** nothing definitive — this may be project-specific. Frame as: *tweaks are valuable while you stay in Claude Design; assume they create cleanup work in Claude Code*. If you know you're handing off, prefer regenerating over tweaking for the final pass.

There is no Claude Desktop embed coming

What we assumed: Claude Design might eventually be embedded directly in Claude Desktop. **What research found:** Anthropic's stated direction is a *deliberate* web-Design + desktop-Code split — handoff is the integration story, not a transitional state. Don't wait for the embed; plan around the handoff workflow as the long-term architecture.

D. The handoff playbook

Putting the capabilities, practices, and gotchas together: when to use Claude Design, and when to switch to Claude Code.

Use Claude Design for: the design system, brand exploration, and a prototype that's "almost good." Stop before pixel-perfecting.

Use Claude Code for: the actual build. Component decomposition, type-safe data flow, real CI, real tests, real deployment. Anything that has to be maintained.

The transition checklist

1. Lock the design system in Claude Design and screenshot the review.
2. Build a prototype that demonstrates the key flows (not all flows).
3. Run the explicit "update result based on tweaks" pass before handoff.
4. Trigger the Claude Code handoff (the project bundle, not the standalone HTML export).
5. In Claude Code: decompose the main HTML into components on day one; don't try to maintain it as a single file.
6. Reserve a portion of your Code/Chat weekly pool for the build phase — Design's pool is gone by now.

The model decision is made for you

Claude Design ships with a model picker, but Opus 4.7 is the trained target. Stay on the default unless you have a specific reason to switch. In Claude Code, model selection is the tunable axis — pick per task (Sonnet for routine work, Opus for hard reasoning, Haiku for cheap iteration).